CERTIK

Security Assessment

BlackCollateral

Jul 17th, 2021

Table of Contents

Summary

Overview

Project Summary

Audit Summary

Vulnerability Summary

Audit Scope

Findings

BCB-01 : Purpose of Function `blackfi()`

BCB-02 : Gas Optimization of Function `transferFrom`

BCB-03 : Logic Issue of `_secondsLeft()`

BCB-04 : `Bought[user]` has Nowhere to Reset

BCB-05 : Lack Checks in Function `_transfer()`

BCB-06 : Logic Issue of `liquidityBalanceWithContract`

BCB-07 : Privileged ownership

BCB-08 : Centralized risk in `addLiquidity`

BCB-09 : Return value not handled

BCB-10 : Confusing Logic of Function `_transfer()`

BCB-11 : Logic Issue of Function `_senderLock()`

BCB-12 : Missing Emit Events

BCB-13 : 3rd party dependencies

BCB-14 : Several Discussions on Logical Issue of `BlackCollateral`

BVB-01 : Privileged ownership

BVB-02 : Missing Important Check

BVB-03 : Several Discussions on Logical Issue of `BurnVault`

BVB-04 : Fund Flow in `BurnVault`

Appendix

Disclaimer

About

Summary

This report has been prepared for BlackCollateral to discover issues and vulnerabilities in the source code of the BlackCollateral project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases given they are currently missing in the repository;
- Provide more comments per each function for readability, especially contracts are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

Overview

Project Summary

Project Name	BlackCollateral
Platform	BSC
Language	Solidity
Codebase	https://github.com/BLACKCOLLATERAL/SmartContracts/tree/main
Commit	41df963bef160e4a0cdd26844f31d753385294ae

Audit Summary

Delivery Date	Jul 17, 2021
Audit Methodology	Manual Review
Key Components	

Vulnerability Summary

Vulnerability Level	Total	Pending	Partially Resolved	Resolved	Acknowledged	Declined
Critical	0	0	0	0	0	0
Major	3	0	0	0	3	0
Medium	0	0	0	0	0	0
Minor	8	0	0	7	1	0
Informational	7	0	0	2	5	0
Discussion	0	0	0	0	0	0

Audit Scope

ID	file	SHA256 Checksum
BVV	BurnVault/BurnVault.sol	55a1034d5ea3537f7d972f8b1e89a9c6f970adba5d53ead4d04817489ec956b4
BUP	Bep20UpgradeableProxy.sol	114704155ceb7d5648fb694d3aecec8f637c457baaec7438b41739f6cbce15c8
BCB	BlackCollateral.sol	2dbf9cd7d6f6410351681859d8e60e460ed1a0c068f5da203639be6fb401dbe3

Understandings

Overview

The Black Collateral Protocol is a decentralized finance (DeFi) token deployed on the Binance smart chain (BSC). Black Collateral Token employs three novel features in its protocol; static rewards for each user, an LP acquisition mechanism and wallet time lock. The static reward (also known as reflection) and LP acquisition mechanisms function as follows:

Each Black Collateral Token transaction is taxed 4% tax fee, 4% burn fee, 1% liquidity fee and 1% treasury fee totalling 10% of the transaction amount. The tax fee is redistributed to all existing holders using a form of rebasing mechanism whilst the liquidity fee is accumulated internally until a sufficient amount of capital has been amassed to perform an LP acquisition. When this number is reached, the total tokens accumulated are split with half being converted to BNB and the total being supplied to the PancakeSwap contract as liquidity.

LP Acquisition

The LP acquisition mechanism can be indirectly triggered by any normal transaction of the token as all transfers evaluate the set of conditions that trigger the mechanism. The main conditions of the mechanism are whether the sender is different than the LP pair and whether the accumulation threshold has been breached. Should these conditions be satisfied, the `swapAndLiquify` function is invoked with the current contract's Black Collateral Token balance.

The `swapAndLiquify` function splits the contract's balance in two halves properly accounting for any truncation that may occur. The first half is swapped to BNB via the PancakeSwap Router using the BLACK-BNB pair and thus temporarily driving the price of the Black Collateral Token down. Afterwards, the resulting BNB balance along with the remaining Black Collateral Token balance are supplied to the BLACK-BNB liquidity pool as liquidity via the Router. The recipient of the LP units is defined as the current owner of the Black Collateral contract.

Static Reward (Reflection)

Balances in the Black Collateral Token system are calculated in one of two ways. The first method, which most users should be familiar with, is a traditional fixed number of units being associated with a user's address. The second method, which is of interest to static rewards, represents a user's balance as a proportion of the total supply of the token.

Whenever a taxed transaction occurs, the 4% meant to be re-distributed to token holders is deducted from the total "proportion" supply resulting in a user's percentage of total supply being increased. Within the

system, not all users are integrated in this system and as such the 4% fee is rewarded to a subset of the total users of the Black Collateral Token. The `owner` of the contract is able to introduce and exclude users from the dynamic balance system at will.

Wallet Time Lock

TIME-LOCK is a novel design that prevents investors from selling their assets in large quantities. This mechanism regulates purchase power by enforcing a time-lock on the holder's wallet, due to which one can't sell or transfer more than 1% of the asset in 24 hours. By enabling the time-lock, asset holders, especially whales, can't manipulate the price significantly, resulting in a long-regulated purchase power increase.

Centralized Risks

The owner of contract `BlackCollateral` has the permission to:

1. change the address that can receive LP tokens,
2. exclude/include addresses from rewards,
3. set `taxFee`, `BurnFee`, `liquidityFee`, `treasuryFee`, `numTokensSellToAddToLiquidity`, `treasuryaddress`
4. forbid any addresses from transactions via `setGreyList`
5. enable `swapAndLiquifyEnabled`,
6. transfer any amount of BNB from the contract to any addresses via function `withdraw()`,
7. use `BEP20UpgradeableProxy` to deploy the contract, and the implementation(Black token) could be updated,

without obtaining the consensus of the community.

The owner of contract `BurnVault` has the permission to:

1. changes the locking period of account from locking state via `_changeUnLockTime()`;
2. set `maxTxAmount`, `lockPeriod`, `black`, `circulatingSupply`;
3. send received black token to any addresses through calling function `burn()`, although the team states that the tokens swapped via Burn Vault will be removed permanently from circulation;

without obtaining the consensus of the community.

Financial Models

Financial models of blockchain protocols need to be resilient to attacks. It needs to pass simulations and verifications to guarantee the security of the overall protocol.

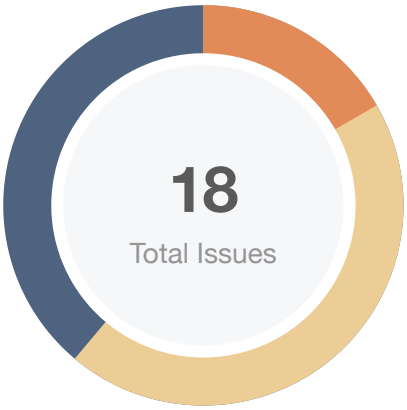
The Black Collateral contract enable a time-lock mechanism to restrict transfers for token holders.

- If sender transfers less than 0.5 % tokens for 2 times within 24 hours, then sender account will be locked for 24 hours.
- If Sender transfers between 0.5 % to 0.75 % of token balance, then sender account will be locked for 12 hours.
- If Sender transfers 0.75 % or more than 0.75 % of token balance, then sender account will be locked for 24 hours.

This will impact the liquidity of this token.

The scope of this audit is not including tokenomics, hence we strongly recommend the team to take serious considerations.

Findings



Critical	0 (0.00%)
Major	3 (16.67%)
Medium	0 (0.00%)
Minor	8 (44.44%)
Informational	7 (38.89%)
Discussion	0 (0.00%)

ID	Title	Category	Severity	Status
BCB-01	Purpose of Function `blackfi()`	Logical Issue	Informational	Resolved
BCB-02	Gas Optimization of Function `transferFrom`	Gas Optimization	Informational	Resolved
BCB-03	Logic Issue of `_secondsLeft()`	Logical Issue	Minor	Resolved
BCB-04	`Bought[user]` has Nowhere to Reset	Logical Issue	Minor	Resolved
BCB-05	Lack Checks in Function `_transfer()`	Logical Issue	Minor	Resolved
BCB-06	Logic Issue of `liquidityBalanceWithContract`	Logical Issue	Minor	Resolved
BCB-07	Privileged ownership	**Centralization / Privilege**	**Major**	**Acknowledged**
BCB-08	Centralized risk in `addLiquidity`	**Centralization / Privilege**	**Major**	**Acknowledged**
BCB-09	Return value not handled	Volatile Code	Informational	Acknowledged
BCB-10	Confusing Logic of Function `_transfer()`	Coding Style	Informational	Acknowledged
BCB-11	Logic Issue of Function `_senderLock()`	Logical Issue	Minor	Resolved
BCB-12	Missing Emit Events	Logical Issue	Minor	Resolved
BCB-13	3rd party dependencies	Control Flow	Minor	Acknowledged

ID	Title	Category	Severity	Status
BCB-14	Several Discussions on Logical Issue of `BlackCollateral`	Logical Issue	Informational	Acknowledged
BVB-01	Privileged ownership	**Centralization / Privilege**	**Major**	**Acknowledged**
BVB-02	Missing Important Check	Logical Issue	Minor	Resolved
BVB-03	Several Discussions on Logical Issue of `BurnVault`	Logical Issue	Informational	Acknowledged
BVB-04	Fund Flow in `BurnVault`	Control Flow	Informational	Acknowledged

BCB-01 | Purpose of Function `blackfi()`

Category	Severity	Location	Status
Logical Issue	Informational	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 948~955	Resolved

Description

The function `blackfi` can be called by anyone. It accepts an uint256 number parameter `tAmount`. The function reduces the Black token balance of the caller by `rAmount`, which is `tAmount` reduces the transaction fee. Then, the function adds `tAmount` to variable `_tFeeTotal`, which represents the contract's total transaction fee. We wish the team could explain more on the purpose of having such functionality.

Alleviation

The team heeded our advice and fixed the issue by removing the function in commit 4f43480eb874f8efb837048548f0323885a77fed.

BCB-02 | Gas Optimization of Function `transferFrom`

Category	Severity	Location	Status
Gas Optimization	Informational	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 902~903	Resolved

Description

It's recommended to check user's allowance before transfer so that in case there is not enough allowance, the function will revert immediately.

```
    function transferFrom(address sender, address recipient, uint256 amount) public
override returns (bool) {
        _transfer(sender, recipient, amount);
        _approve(sender, _msgSender(), _allowances[sender][_msgSender()].sub(amount,
"BEP20: transfer amount exceeds allowance"));
        return true;
    }
```

Recommendation

Consider changing the code like below:

```
    function transferFrom(address sender, address recipient, uint256 amount) public
override returns (bool) {
        _approve(sender, _msgSender(), _allowances[sender][_msgSender()].sub(amount,
"BEP20: transfer amount exceeds allowance"));
        _transfer(sender, recipient, amount);
        return true;
    }
```

Alleviation

The team heeded our advice and fixed the issue in commit 4f43480eb874f8efb837048548f0323885a77fed.

BCB-03 | Logic Issue of `_secondsLeft()`

Category	Severity	Location	Status
Logical Issue	Minor	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1009~1024	Resolved

Description

When bought[sender] is neither 50 nor 60, it goes to the third case. But the expression surely will get an underflow result because if the sender's account is frozen, `_boughtBlock[sender] > block.timestamp`.

Even if it is designed for calculation, the correct left seconds should be `_boughtBlock[sender] - block.timestamp` rather than some number from the calculation with `_blockFreeze` involved.

Recommendation

Consider checking the logic and fixing this issue.

Alleviation

The team heeded our advice and fixed the issue in commit 4f43480eb874f8efb837048548f0323885a77fed.

BCB-04 | `Bought[user]` has Nowhere to Reset

Category	Severity	Location	Status
Logical Issue	Minor	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1003~1007	Resolved

Description

The state `bought[user]` can only be changed to 50 or 60. There is nowhere to reset, which may cause wrong value in function `_secondsLeft()`.

Recommendation

Consider checking the logic and fixing the issue.

Alleviation

The team heeded our advice and fixed the issue by removing this variable and recording lock time with variable `_boughtBlock`. Modifications can be found in commit 4f43480eb874f8efb837048548f0323885a77fed.

BCB-05 | Lack Checks in Function `_transfer()`

Category	Severity	Location	Status
Logical Issue	Minor	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1041	Resolved

Description

It is recommended to check if `sender == recipient` in function `_transfer()` as it's meaningless for users to transfer to themselves.

Recommendation

Consider changing the code like below.

```
function _transfer(address sender, address recipient, uint256 amount) private {
    require(sender != address(0), "BEP20: transfer from the zero address");
    require(recipient != address(0), "BEP20: transfer to the zero address");
    require(sender != recipient, "BEP20: transfer to self");
```

Alleviation

The team heeded our advice and fixed the issue in commit 4f43480eb874f8efb837048548f0323885a77fed.

BCB-06 | Logic Issue of `liquidityBalanceWithContract`

Category	Severity	Location	Status
Logical Issue	Minor	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1057~1069	Resolved

Description

As is shown in the code, `liquidityBalanceWithContract` will be assigned the value of `numTokensSellToAddToLiquidity` when the conditions are met. So it's meaningless to set `liquidityBalanceWithContract = min(liquidityBalanceWithContract, _maxLiquidityAmount)`.

Recommendation

Consider checking the logic and fixing the issue.

Alleviation

The team heeded our advice and fixed the issue in commit 4f43480eb874f8efb837048548f0323885a77fed.

BCB-07 | Privileged ownership

Category	Severity	Location	Status
Centralization / Privilege	Major	contracts/BLACKCOLLATERAL/BlackCollateral.sol	Acknowledged

Description

The owner of contract `BlackCollateral` has the permission to:

1. change the address that can receive LP tokens,
2. exclude/include addresses from rewards,
3. set `taxFee`, `BurnFee`, `liquidityFee`, `treasuryFee`, `numTokensSellToAddToLiquidity`, `treasuryaddress`
4. forbid any addresses from transactions via `setGreyList`
5. enable `swapAndLiquifyEnabled`,
6. transfer any amount of BNB from the contract to any addresses via function `withdraw()`,
7. use `BEP20UpgradeableProxy` to deploy the contract, and the implementation(Black token) could be updated,

without obtaining the consensus of the community.

Recommendation

Renounce ownership when it is the right timing, or gradually migrate to a timelock plus multisig governing procedure and let the community monitor in respect of transparency considerations.

Alleviation

The team acknowledged the issue and stated:

1. Multisig deployment: Currently, the contract owner holds all the essential privileges. Based on the recommendation from the BSC Technical Architects, we are awaiting native GNOSIS support to implement both proxies and smart contract implementations to multisig enabled addresses. An approved set of administrators will govern Multisig for BLACK COLLATERAL contracts. Any privileged access such as changing the contract state or contract upgrades will require approval from a minimum number of approved multisig parties. Through the multisig approach, BLACK contracts eliminate the single point of failure of EOA owned contracts.

2. The BLACK Collateral Team will leverage the https://snapshot.org forum to add transparency to the token holders regarding governance and smart contract change proposals.

3. Moreover, we are not an anonymous company or an individual developing this project. Boson Labs is a global R&D organization registered in both the USA and India, serving the crypto community for over four years. Developers, Leads, Architects, and Digital Marketing Consultants working for the Black Collateral project from Boson Labs are listed in the BLACK Collateral Websites for transparency.

BCB-08 | Centralized risk in `addLiquidity`

Category	Severity	Location	Status
Centralization / Privilege	Major	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1171	Acknowledged

Description

```
// add the liquidity
uniswapV2Router.addLiquidityETH{value: ethAmount}(
    address(this),
    tokenAmount,
    0, // slippage is unavoidable
    0, // slippage is unavoidable
    owner(),
    block.timestamp
);
```

The `addLiquidity` function calls the `uniswapV2Router.addLiquidityETH` function with the `to` address specified as `owner()` for acquiring the generated LP tokens from the `BLACK-BNB` pool. As a result, over time the `_owner` address will accumulate a significant portion of LP tokens. If the `_owner` is an EOA (Externally Owned Account), mishandling of its private key can have devastating consequences to the project as a whole.

Recommendation

We advise the `to` address of the `uniswapV2Router.addLiquidityETH` function call to be replaced by the contract itself, i.e. `address(this)`, and to restrict the management of the LP tokens within the scope of the contract's business logic. This will also protect the LP tokens from being stolen if the `_owner` account is compromised. In general, we strongly recommend centralized privileges or roles in the protocol to be improved via a decentralized mechanism or via smart-contract based accounts with enhanced security practices, f.e. Multisignature wallets.

Indicatively, here are some feasible solutions that would also mitigate the potential risk:

- Time-lock with reasonable latency, i.e. 48 hours, for awareness on privileged operations;
- Assignment of privileged roles to multi-signature wallets to prevent single point of failure due to the private key;
- Introduction of a DAO / governance / voting module to increase transparency and user involvement.

Alleviation

The team acknowledged the issue and stated:

"Currently, the LP tokens are held by the owner of the contract. It gives us more flexibility regarding LP handling to manage future DEX (PancakeSwap) version changes, router address changes, or migrate to a better exchange in the future. Moreover, the team is not anonymous and from the reputed global R&D institution with a public presence hence we believe the overall risk is very minimal."

BCB-09 | Return value not handled

Category	Severity	Location	Status
Volatile Code	Informational	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1171	Acknowledged

Description

The return values of function `addLiquidityETH` are not properly handled.

```
        uniswapV2Router.addLiquidityETH{value: ethAmount}(
            address(this),
            tokenAmount,
            0, // slippage is unavoidable
            0, // slippage is unavoidable
            owner(),
            block.timestamp
        );
```

Recommendation

We recommend using variables to receive the return value of the functions mentioned above and handle both success and failure cases if needed by the business logic.

Alleviation

The issue has been acknowledged by the team. No code change was made as it will not impact the overall functionality of the contract.

BCB-10 | Confusing Logic of Function `_transfer()`

Category	Severity	Location	Status
Coding Style	Informational	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1071~1128	Acknowledged

Description

Actually, functions with the name `_transfer*()` in contract `BlackCollateral` have very similar logic.

They can be simplified by introducing several parameters:

1. `ifSenderExcluded`
2. `ifReceipientExcluded`
3. `ifOnFee`

Many functions' implementations are totally different from the comments above them.

Recommendation

Consider optimizing this part to get the logic clearer to users.

Alleviation

The issue has been acknowledged by the team. No code change was made as it will not impact the overall functionality of the contract.

BCB-11 | Logic Issue of Function `_senderLock()`

Category	Severity	Location	Status
Logical Issue	Minor	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1206~1216	Resolved

Description

Condition 2 and 3 has nothing to do with condition 1 in below codes. It doesn't require `blacklisted[sender] <= 0`. And `blacklisted[sender]` should be cleared here so that the function will not revert after one transfer of value smaller than 0.5%.

```
if(amount < pointfive  ){
        blacklisted[sender]=blacklisted[sender] + 1;
        if(blacklisted[sender]==1)
            blacklistedtime[sender]= block.timestamp;
        if(block.timestamp < blacklistedtime[sender] + _secondLockPeriod ){
                if(blacklisted[sender] >= 2){
                _boughtBlock[sender] = block.timestamp + _secondLockPeriod ;
                bought[sender]=60;
                boughttime[sender]=block.timestamp;
                blacklisted[sender]=0;
                }
        }else{
                blacklisted[sender]=1;
                blacklistedtime[sender]=block.timestamp;
        }

 }else if(amount >= pointfive && amount < pointsevenfive ){
            require(blacklisted[sender]<=0,"Current sender is in frozen state !");
            _boughtBlock[sender] = block.timestamp + _firstLockPeriod;
            bought[sender]=50;
            boughttime[sender]=block.timestamp;
 }else if (amount >= pointsevenfive){
            require(blacklisted[sender]<=0,"Current sender is in frozen state !");
            _boughtBlock[sender] = block.timestamp + _secondLockPeriod ;
            bought[sender]=60;
            boughttime[sender]=block.timestamp;
 }
```

Recommendation

Consider checking the logic and fixing the issue.

Alleviation

[Black Collateral]:

The team heeded our advice and update code in commit 4f43480eb874f8efb837048548f0323885a77fed.

[CertiK Response]:

Regard to the modifications in commit 4f43480eb874f8efb837048548f0323885a77fed, function `_senderTimeLock` is introduced to help lock users.

As is shown in the code, if a user has one transfer below 0.5% within 24 hours, he cannot transfer over 0.5%. From some aspect, the user has already been locked without recorded twice.

[Black Collateral]:

The team stated that it's designed to restrict users from transferring more than 1% of their assets. So the code meets their expectations.

For example:

1.If holder A transfers 0.4% tokens first, his account will not get locked, and the transaction will go through. If the holder tries again to transfer 0.9% of his wallet balance, the protocol does not allow this transfer because of the total transfer value, i.e .0.4 +0.9 > 1%. At the same time, holder A can transfer below 0.5 % of tokens within 24 hours. 2.If holder A transfers 0.4 % tokens first, his account will not get locked, and the transaction will go through successful. However, if holder A receives tokens from holder B, which is more than 0.5 % of A's holding, Holder A will get locked. More details about the receiver lock are explained in the diagram and demonstration section below

More details in https://docs.blackcollateral.com/core/flowchart.

********** DEMONSTRATION SECTION **********

In general, the protocol enforces two levels of precheck restrictions. Any point in time,

1. Users can't make more than two transactions from a wallet in 24 hours
2. Users can't transfer more than 1% of the wallet balance in 24 hours

Let's look at how black protocol works by taking the following example.

```
Sam has 20000 BLACK assets in his wallet
```

SCENARIO 1: Sam initiates a transaction to sell or transfer 1000 BLACK ASSETS The transfer amount is 5% of his wallet balance, exceeding the acceptable 1% daily limit. As a result, the protocol will not allow the transaction

SCENARIO 2: Sam initiates a transaction to sell or transfer 100 BLACK ASSETS The transfer amount is 0.5% of the wallet balance and is within the acceptable 1% daily limit. Also, the daily transaction count is zero, and the wallet is not locked. As a result, the platform approves the transaction after performing the following four protocol operations

- Accumulates 1% (1 BLACK token) of the transaction amount in Burn Vault.
- Send 1% (1 BLACK token) of the transaction amount to the exchange liquidity pool.
- Burn 4% (4 BLACK tokens) of the transaction amount forever from circulation.
- Distributes 4% (4 BLACK tokens) of the transaction amount as rewards equally among holders.

Once the transaction is complete, the protocol

- Increases the daily transaction counter by 1
- Lock the Wallet for 12 hours since it reaches 50% of the 1% limit

SCENARIO 3: Sam initiates a transaction to sell or transfer 200 BLACK ASSETS The transfer amount is 1% of the wallet balance and is within the acceptable threshold of a 1% daily limit. As a result, the platform approves the transaction after performing the following six protocol operations

- Increases the daily transaction counter by 1
- Lock the Wallet for 24 hours since it reaches 100% of the 1% limit
- Accumulates 1% (2 BLACK tokens) of the transaction amount in Burn Vault.
- Send 1% (2 BLACK tokens) of the transaction amount to the exchange liquidity pool.
- Burn 4% (8 BLACK tokens) of the transaction amount forever from circulation.
- Distributes 4% (8 BLACK tokens) of the transaction amount as rewards equally among holders.

SCENARIO 4: Let's look at the situation when the investor initiates three transactions of 50 BLACK ASSETS, each consecutively

- The 1st transfer amount is 0.25% of the wallet balance and is within the acceptable 1% daily threshold. The platform approves the transaction and increases the daily transaction counter by 1
- The 2nd transfer amount is also 0.25% of the wallet balance and is within the acceptable 1% daily threshold. The platform approves the transaction after increasing the daily transaction counter by one that makes two transactions in 24 hours and the wallet for 12 hours since it reaches a 50% transfer limit
- When the 3rd transfer kicks in, the wallet is already in the locked state, and the total daily transaction counter already reached its maximum threshold (2 TX). Hence the protocol will not approve the 3rd one, and the user needs to wait for another 24 hours before initiating a transfer.

BCB-12 | Missing Emit Events

Category	Severity	Location	Status
Logical Issue	Minor	contracts/BLACKCOLLATERAL/BlackCollateral.sol	Resolved

Description

Functions that affect the status of sensitive variables should be able to emit events as notifications to customers.

For example:

- `withdraw()`
- `_setTreasuryAddress()`

Recommendation

Consider adding events for sensitive actions, and emit it in the function.

```
function withdraw(address payable _target,uint256 _amount)external  onlyOwner {
   require(address(this).balance > 0,"Not enough BNB balance");
   _target.transfer(_amount);
   emit Withdraw(_target, _amount);
}
```

Alleviation

The team heeded our advice and fixed the issue in commit 4f43480eb874f8efb837048548f0323885a77fed.

BCB-13 | 3rd party dependencies

Category	Severity	Location	Status
Control Flow	Minor	contracts/BLACKCOLLATERAL/BlackCollateral.sol	Acknowledged

Description

The contract is serving as the underlying entity to interact with third party PancakeSwap protocols. The scope of the audit would treat those 3rd party entities as black boxes and assume its functional correctness. However in the real world, 3rd parties may be compromised that led to assets lost or stolen.

Recommendation

We understand that the business logic of the BlackCollateral protocol requires the interaction PancakeSwap protocol for adding liquidity to BLACK-BNB pool and swap tokens. We encourage the team to constantly monitor the statuses of those 3rd parties to mitigate the side effects when unexpected activities are observed.

Alleviation

The team acknowledged the issue and stated:

A robust plan in place to regularly monitor the third party libraries for any changes and make necessary code revisions. We believe this arrangement will help us avoid any third party library related issues.

BCB-14 | Several Discussions on Logical Issue of `BlackCollateral`

Category	Severity	Location	Status
Logical Issue	Informational	contracts/BLACKCOLLATERAL/BlackCollateral.sol: 1071~1128	Acknowledged

Description

According to the `README.md` file and code comments, we have several questions about the contract logic:

1. Rule of charging fees, we cannot find the specific rule of charging fees during transfer.

Like transfer to `burnVault` is free from fee,

```
    else if(recipient == burnVault){
        _transferTorewardExcluded(sender, recipient, amount);
    }
```

but transfer to `uniswapV2Pair` is not free from fee, except the admin,

```
    if(recipient  == uniswapV2Pair){
        if(sender==address(this)||sender==owner())
        {
        _transferFromrewardExcluded(sender, recipient, amount);
        }
        else{
            require(amount<= maxTxAmount , 'maxTxAmount should be less than or equals 1 %
');
        _transferFromrewardExcludedPCS(sender,recipient,amount);
        }
    }
```

2. Only function `_transferStandard()` can lock users, is this designed intentionally?
3. The concept `stakingAddress` is mentioned in the code, transfers to or from staking addresses will be charged fees. They are set by the owner of the contract. So what kind of addresses will be set `stakingAddress`?
4. We can see the default liquidityFee is 1% from the code, but the documentation states as 0.5%: "AUTOMATED LIQUIDITY POOL (ALP) - prevents exchange liquidity fluctuation to a greater extent by locking 0.5% of every transaction back in the liquidity pool."
5. What is the usage of treasuryFee?

Could you please explain to above questions?

Alleviation

Below are explanations given by the team.

1. Fees collection for different types of wallets are captured in the "Protocol Matrix" workbook
2. The Locking scenarios are documented in the "Protocol Matrix" workbook
3. Refer to the "Protocol Matrix" workbook to understand the staking metrics and locking dynamics.
4. ALP 0.5 vs 1 : Kindly refer to the Whitepaper that has all the ALP logics detailed for reference. " AUTOMATED LIQUIDITY POOL (ALP) MECHANISM prevents exchange liquidity fluctuation to a greater extent by locking a portion of every transaction back in the liquidity pool, enhancing the overall market price of the crypto asset."
5. What is the usage of treasuryFee?
   "The tokens collected as part of the Treasury fees will be converted to BNB and sent to BurnVault contract periodically. Subsequently the accumulated BNB will be swapped for BLACK tokens to give to users when they exchange their Black tokens via BurnVault contract. We have not automated this now, because we need to check the market value / pricing of Black tokens before we exchange those Black tokens for BNB to avoid price depreciation. Additionally, the exact BLACK assets will be sent to burn address"

BVB-01 | Privileged ownership

Category	Severity	Location	Status
Centralization / Privilege	Major	contracts/BLACKCOLLATERAL/BurnVault/BurnVault.sol	Acknowledged

Description

The owner of contract `BurnVault` has the permission to:

1. changes the locking period of account from locking state via `_changeUnLockTime()`;
2. set `maxTxAmount`, `lockPeriod`, `black`, `circulatingSupply`;
3. send received black token to any addresses through calling function `burn()`, although the team states that the tokens swapped via Burn Vault will be removed permanently from circulation;

```
function burn(address to,uint256 amount) public onlyOwner{
   black.transfer(to ,amount);
}
```

without obtaining the consensus of the community.

Recommendation

Consider checking the logic and fixing the issue.

Renounce ownership when it is the right timing, or gradually migrate to a timelock plus multisig governing procedure and let the community monitor in respect of transparency considerations.

Alleviation

The team acknowledged the issue and stated:

1. Multisig deployment: Currently, the contract owner holds all the essential privileges. Based on the recommendation from the BSC Technical Architects, we are awaiting native GNOSIS support to implement both proxies and smart contract implementations to multisig enabled addresses. An approved set of administrators will govern Multisig for BLACK COLLATERAL contracts. Any privileged access such as changing the contract state or contract upgrades will require approval from a minimum number of approved multisig parties. Through the multisig approach, BLACK contracts eliminate the single point of failure of EOA owned contracts.

2. The BLACK Collateral Team will leverage the https://snapshot.org forum to add transparency to the token holders regarding governance and smart contract change proposals.

3. Moreover, we are not an anonymous company or an individual developing this project. Boson Labs is a global R&D organization registered in both the USA and India, serving the crypto community for over four years. Developers, Leads, Architects, and Digital Marketing Consultants working for the Black Collateral project from Boson Labs are listed in the BLACK Collateral Websites for transparency.

BVB-02 | Missing Important Check

Category	Severity	Location	Status
Logical Issue	Minor	contracts/BLACKCOLLATERAL/BurnVault/BurnVault.sol	Resolved

Description

In function `transfer()`, it does not check the balance is enough to transfer `_tokenAmount` before calling of `sendBackBNB`.

Recommendation

Consider adding below checking:

```
  require(getBurnVaultBNBBalance() > _tokenAmount, "BurnVault has not enough BNB");
...
  sendBackBNB(_tokenAmount);
```

Alleviation

The team heeded our advice and fixed the issue in commit 380cf0498a0e4a51da7f4a246ad2bfc5016d7300.

BVB-03 | Several Discussions on Logical Issue of `BurnVault`

Category	Severity	Location	Status
Logical Issue	Informational	contracts/BLACKCOLLATERAL/BurnVault/BurnVault.sol: 603~610	Acknowledged

Description

According to the `README.md` file and code comments, we have several questions about the contract logic:

1. The function will revert when `balance >= maxTxAmount`. Since the transfer has not been done here, it is recommended to assign `_tokenAmount = Min(maxTxAmount - senderBurnBalance[_from], _tokenAmount)`.
2. The function could be called to synchro the value of `circulatingSupply` with `black.totalSupply()`. It could slightly change the rate `tokenPerBNB`.

Recommendation

Consider to take a consideration on these above quesitons.

Alleviation

The team acknowledged the issue and stated:

This is acknowledged, As per the recommendation, the totalSupply is retrieved from Black token itself.

BVB-04 | Fund Flow in `BurnVault`

Category	Severity	Location	Status
Control Flow	Informational	contracts/BLACKCOLLATERAL/BurnVault/BurnVault.sol	Acknowledged

Description

According to the `README.md` file, the Burn Vault accumulates the transaction fees in a separate contract address where users may choose to cash out at any point in time, similar to the Federal Gold Reserve, without impacting the market price.

We can see the Burn Vault is used to swap black token for BNB. But we can not find any logic to align back the above description.

Which contract is the separate contract to accumulates the transaction fees? Where is the BNB from for the usage of swap? Could you please explain to the flow of funds?

Alleviation

Below are explanations given by the team.

The tokens collected as part of the Treasury fees will be converted to BNB and sent to BurnVault contract periodically. Subsequently the accumulated BNB will be swapped for BLACK tokens to give to users when they exchange their Black tokens via BurnVault contract. We have not automated this now, because we need to check the market value / pricing of Black tokens before we exchange those Black tokens for BNB to avoid price depreciation. Additionally, the exact BLACK assets will be sent to burn address.

Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion on how block.timestamp works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invoke-able by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without CertiK's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

About

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

CERTIK